

PRUEBA DE EVALUACIÓN CONTINUA

María Soriano Palacios

1. Acelerómetro ADXL335: Obtener la aceleración e imprimirla por pantalla

Explicación

El objetivo de este ejercicio es obtener los valores de aceleración proporcionados por el acelerómetro ADXL335 en sus tres ejes, X, Y y Z, e imprimirlos por pantalla.

El programa inicializa la pantalla **LCD**, el **Serial Monitor** y el **acelerómetro**.

Después, cada segundo lee los valores de los tres ejes mediante la función '*getXYZ()*'. Los datos obtenidos se imprimen en el Serial Monitor y también se muestran en la pantalla LCD.

Para mejorar la organización del código, se han creado funciones independientes para leer el sensor, imprimir los datos por SM y actualizar la pantalla LCD. Además, se utiliza *millis* en lugar de *delay* para evitar bloquear la ejecución del programa.

Resultado

Al ejecutar el programa, la pantalla LCD muestra los valores de X, Y y Z. Estos valores también aparecen en el Serial Monitor (SM). Si cambia la orientación o movimiento del sensor, los valores de los ejes varían.



Serial output:

```
X: 20 | Z: 419
X: 118 | Y: 15 | Z: 419
X: 222 | Y: 5 | Z: 419
X: 69 | Y: 0 | Z: 419
X: 241 | Y: 25 | Z: 419
X: 87 | Y: 72 | Z: 419
X: 49 | Y: 143 | Z: 419
X: 7 | Y: 41 | Z: 419
```

Código

Exercise_1__ADXL335_Accelerometer.ino

```
1  /*
2   Exercise 1 - ADXL335 Accelerometer
3
4   This program reads the acceleration values from the X, Y and Z axes
5   using the ADXL335 accelerometer. The values are displayed on the LCD
6   screen and printed to the Serial Monitor every second.
7
8   The code uses functions to keep the program organized and millis()
9   instead of delay() to avoid blocking the main loop
10  */
11
12  #include <Wire.h>           // I2C communication library
13  #include <LiquidCrystal_I2C.h> // LCD display library
14  #include "ADXL335.h"       // ADXL335 accelerometer library
15
16  // accelerometer pins
17  #define X_AXIS_PIN A8
18  #define Y_AXIS_PIN A9
19  #define Z_AXIS_PIN A10
20
21  // hardware objects: accelerometer and lcd
22  ADXL335 accelerometer;
23  LiquidCrystal_I2C lcd(0x3F, 16, 2);
24
25  // time interval (in milliseconds) between sensor readings
26  const unsigned long READ_INTERVAL = 1000;
27  // stores last time sensor was read
28  unsigned long previousMillis = 0;
29
30  // accelerometer readings
31  int x = 0;
32  int y = 0;
33  int z = 0;
34
35  // last displayed values, used to avoid unnecessary LCD refreshes
36  //    -> init to -1 so first real reading always triggers an update
37  int lastX = -1;
38  int lastY = -1;
39  int lastZ = -1;
40
41  void setup()
42  {
43    // init Serial communication
44    Serial.begin(9600);
45  }
```

```
46 // init the LCD display
47 lcd.init();
48 lcd.backlight();
49 lcd.clear();
50
51 // init accelerometer
52 accelerometer.begin();
53
54 // startup message
55 lcd.setCursor(0, 0);
56 lcd.print("ADXL335 ready");
57 }
58
59 void loop()
60 {
61     // get current time since program started
62     unsigned long currentMillis = millis();
63
64     // check if defined time interval has passed
65     if (currentMillis - previousMillis >= READ_INTERVAL)
66     {
67         // update last exec time
68         previousMillis = currentMillis;
69
70         // read and display acceleration values (update lcd ONLY if values have changed)
71         readAccelerometer();
72         printSerial();
73         updateLCDIfNeeded();
74     }
75 }
76
77 void readAccelerometer()
78 {
79     // reads values from axes
80     accelerometer.getXYZ(&x, &y, &z);
81 }
82
83 void printSerial()
84 {
85     // prints readings to Serial Monitor
86     Serial.print("X: ");
87     Serial.print(x);
88     Serial.print(" | Y: ");
89     Serial.print(y);
90     Serial.print(" | Z: ");
91     Serial.println(z);
92 }
```

```
93
94 void updateLCDIfNeeded()
95 {
96     // ONLY update the LCD if values have changed
97     // -> this avoids flickering
98     if (x != lastX || y != lastY || z != lastZ)
99     {
100         // clears previous values
101         lcd.clear();
102
103         // displays x (first row)
104         lcd.setCursor(0, 0);
105         lcd.print("X:");
106         lcd.print(x);
107
108         // displays y (first row, right side)
109         // -> left side: columns 0-7
110         // -> right side: columns 8-15
111         lcd.setCursor(8, 0);
112         lcd.print("Y:");
113         lcd.print(y);
114
115         // displays z (second row)
116         lcd.setCursor(0, 1);
117         lcd.print("Z:");
118         lcd.print(z);
119
120         // stores current values
121         lastX = x;
122         lastY = y;
123         lastZ = z;
124     }
125 }
```

2. Sensor de temperatura y humedad DHT22: Obtener la humedad e imprimirla junto a la temperatura por pantalla

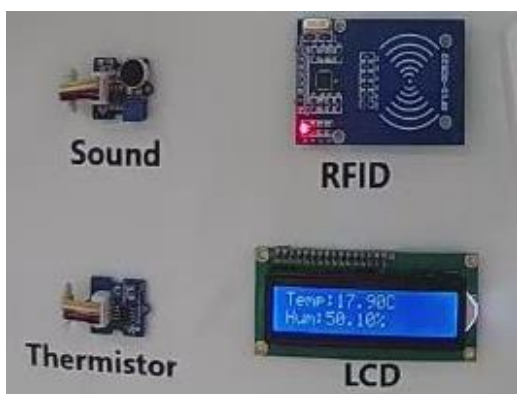
Explicación

El objetivo de este ejercicio es obtener la temperatura y humedad mediante el sensor **DHT22** y mostrar ambos valores tanto en la pantalla LCD como en el Serial Monitor. El sensor DHT22 nos permite **medir la temperatura y humedad relativa** mediante una **salida digital**. Según sus características, requiere un intervalo mínimo **de 2 segundos** entre lecturas para funcionar correctamente, ya que su frecuencia de muestreo es de 0,5 Hz.

El programa inicializa el sensor DHT22, la pantalla LCD y el Serial Monitor. Cada 2 segundos se realiza una nueva lectura utilizando *millis* en lugar de *delay*, lo que evita bloquear la ejecución del programa. Después, se comprueba que los valores obtenidos sean válidos mediante '*isnan()*', ya que el sensor puede devolver valores NaN si ocurre un error de lectura. Si la lectura es correcta, los valores de temperatura y humedad se muestran en la pantalla LCD y en el Serial Monitor (SM). Además, la pantalla LCD solo se actualiza cuando los valores cambian, optimizando la actualización de pantalla para evitar parpadeos (flickering). Si ocurre un error, se muestra un mensaje indicando que se debe revisar el sensor.

Resultado

La pantalla LCD muestra la **temperatura en grados Celsius** y la **humedad en porcentaje**. Los valores se actualizan cada 2 segundos. Si ocurre un error en la lectura, se muestra un mensaje de error.



```
Te | Humidity: 50.10 %  
  
Temperature: 17.80 C | Humidity: 50.00 %  
Temperature: 17.80 C | Humidity: 50.00 %  
Temperature: 17.90 C | Humidity: 50.10 %
```

Código

```
1  /*
2   Exercise 2 - Temperature & Humidity (DHT22)
3
4   This program reads temperature and humidity values from the DHT22 sensor
5   and displays them on the LCD and Serial Monitor every 2 seconds.
6  */
7
8  #include <Wire.h>           // I2C communication library
9  #include <LiquidCrystal_I2C.h> // LCD display library
10 #include "DHT.h"           // DHT sensor library
11
12 // DHT configuration
13 #define DHT22_PIN 12
14 #define DHTTYPE DHT22
15
16 DHT dht(DHT22_PIN, DHTTYPE);
17
18 // LCD (address 0x3F, column 16, row 2)
19 LiquidCrystal_I2C lcd(0x3F, 16, 2);
20
21 // timing (DHT22 NEEDS ~2 seconds between readings) !
22 const unsigned long READ_INTERVAL = 2000;
23 // stores last time sensor was read
24 unsigned long previousMillis = 0;
25
26 // dht values
27 float temperature = 0;
28 float humidity = 0;
29
30 // last values, used to avoid unnecessary LCD refreshes and flickering
31 // -> init to -1000 so first reading always triggers an update
32 float lastTemp = -1000;
33 float lastHum = -1000;
34
35 void setup()
36 {
37     // init Serial communication
38     Serial.begin(9600);
39
40     // init the LCD display
41     lcd.init();
42     lcd.backlight();
43     lcd.clear();
44
45     // init dht
46     dht.begin();
```

```
47
48 // startup message
49 lcd.setCursor(0, 0);
50 lcd.print("DHT22 ready");
51 }
52
53 void loop()
54 {
55 // get current time since program started
56 unsigned long currentMillis = millis();
57
58 // check if defined time interval (2s) has passed
59 if (currentMillis - previousMillis >= READ_INTERVAL)
60 {
61 // updates last exec time
62 previousMillis = currentMillis;
63
64 // if TRUE -> read and display values (update lcd ONLY if values have changed)
65 if (readDHT())
66 {
67 printSerial();
68 updateLCDIfNeeded();
69 }
70 // else -> show error message
71 else
72 {
73 handleSensorError();
74 }
75 }
76 }
77
78 //returns TRUE (OK) / FALSE (NaN)
79 bool readDHT()
80 {
81 // reads temperature and humidity
82 temperature = dht.readTemperature();
83 humidity = dht.readHumidity();
84
85 // validates readings (dht returns NaN if error)
86 if (isnan(temperature) || isnan(humidity))
87 {
88 return false;
89 }
90
91 return true;
92 }
```

```
94 void printSerial()
95 {
96     // print reading to Serial Monitor
97     Serial.print("Temperature: ");
98     Serial.print(temperature);
99     Serial.print(" C | Humidity: ");
100    Serial.print(humidity);
101    Serial.println(" %");
102 }
103
104 void updateLCDIfNeeded()
105 {
106     // ONLY update LCD if values have changed
107     // -> this avoids flickering
108     if (temperature != lastTemp || humidity != lastHum)
109     {
110         // clears previous values
111         lcd.clear();
112
113         // displays temperature (first row)
114         lcd.setCursor(0, 0);
115         lcd.print("Temp:");
116         lcd.print(temperature);
117         lcd.print("C");
118
119         // displays humidity (second row)
120         lcd.setCursor(0, 1);
121         lcd.print("Hum:");
122         lcd.print(humidity);
123         lcd.print("%");
124
125         // stores current values
126         lastTemp = temperature;
127         lastHum = humidity;
128     }
129 }
130
131 // Handle sensor error
132 void handleSensorError()
133 {
134     // debug message: dht reading error
135     Serial.println("Error: DHT22 reading failed");
136
137     // clears previous values
138     lcd.clear();
139     // displays msg "Sensor error" (first row)
140     lcd.setCursor(0, 0);
141     lcd.print("Sensor error");
142     // displays msg "Check DHT22" (second row)
143     lcd.setCursor(0, 1);
144     lcd.print("Check DHT22");
145 }
```

3. Barómetro BMP085: Obtener la altitud a partir de la lectura de presión e imprimirla por pantalla. Modificar la presión obtenida en Pascales y convertirla a Atmosferas

Explicación

El objetivo de este ejercicio es utilizar el sensor barométrico **BMP085** para obtener la **presión atmosférica**, calcular la altitud estimada a partir de dicha presión y convertir la presión medida en Pascales a atmósferas.

El sensor BMP085 permite medir la presión y la temperatura mediante comunicación **I2C** con la placa Arduino. A partir de la presión atmosférica es posible estimar la altitud, ya que ambas magnitudes están relacionadas físicamente.

El programa inicializa el barómetro, la pantalla LCD I2C y la comunicación serie. Cada 2 segundos se realiza la lectura de la temperatura y la presión en Pascales. Posteriormente, se convierte a atmósferas y se calcula la altitud utilizando las funciones proporcionadas por la librería del sensor.

Los resultados se muestran en el Serial Monitor y en la pantalla LCD. Para mejorar la organización del código y su legibilidad, se han utilizado funciones independientes encargadas de la lectura del sensor, la impresión de los datos y la actualización de la pantalla. Además, utilizamos *millis* en lugar de *delay* para controlar el intervalo entre lecturas sin bloquear la ejecución del programa.

Conversión usada

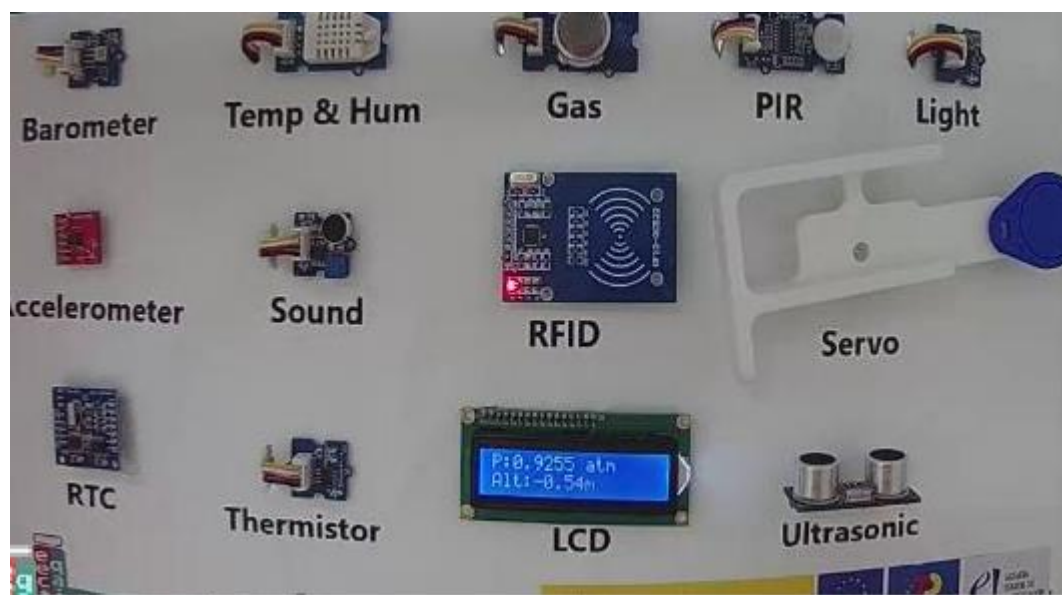
$$1 \text{ atm} = 101325 \text{ PA}$$

$$\text{Presión atm} = \text{presión PA} / 101325$$

Resultado

Al ejecutar el programa se obtiene la **presión atmosférica** en **Pascales**, su equivalente en **atmósferas** y la **altitud** estimada en **metros**. Estos valores se actualizan periódicamente y se muestran en el SM y en la pantalla LCD, permitiendo observar cómo varían en función del entorno.

```
Tempe: 93781.00 Pa | Pressure: 0.9255 atm | Altitude: -0.27 m
ssure: 0.9255 atm | Altitude: -1.08 m
Temperature: 17.40 C | Pressure: 93781.00 Pa | Pressure: 0.9255 atm | Altitude: -0.27 m
ssure: 0.9255 atm | Altitude: -1.08 m
Temperature: 17.40 C | Pressure: 93781.00 Pa | Pressure: 0.9255 atm | Altitude: -0.27 m
Temperature: 17.40 C | Pressure: 93784.00 Pa | Pressure: 0.9256 atm | Altitude: 0.27 m
Temperature: 17.40 C | Pressure: 93793.00 Pa | Pressure: 0.9257 atm | Altitude: 0.81 m
Temperature: 17.40 C | Pressure: 93784.00 Pa | Pressure: 0.9256 atm | Altitude: 0.27 m
```



Código

```
1  /*
2  Exercise 3 - BMP085 Barometer
3
4  This program reads temperature and pressure from the BMP085 barometer,
5  calculates altitude from pressure, converts pressure from Pascals to atm,
6  and displays both on the LCD and Serial Monitor the results every 2 seconds.
7
8  */
9
10 #include <Wire.h>           // I2C communication library
11 #include <LiquidCrystal_I2C.h> // LCD display library
12 #include "BMP085.h"        // BMP085 barometer library
13
14 // hardware objects
15 LiquidCrystal_I2C lcd(0x3F, 16, 2);
16 BMP085 barometer;
17
18 // timing configuration (2sec)
19 const unsigned long READ_INTERVAL = 2000;
20 unsigned long previousMillis = 0;
21
22 // sensor values
23 float temperature = 0;
24 float pressurePa = 0;
25 float pressureAtm = 0;
26 float altitude = 0;
27
28 // conversion constant: 1 atmosphere = 101325 Pascals
29 const float PA_PER_ATM = 101325.0;
30
31 void setup()
32 {
33     // init Serial Monitor
34     Serial.begin(9600);
35
36     // init LCD
37     lcd.init();
38     lcd.backlight();
39     lcd.clear();
40
41     // init BMP085 sensor
42     barometer.init();
43
44     // startup message
45     lcd.setCursor(0, 0);
46     lcd.print("BMP085 ready");
```

```
46     lcd.print("BMP085 ready");
47 }
48
49 void loop()
50 {
51     // get current time since program started
52     unsigned long currentMillis = millis();
53
54     // check if defines time interval has passed
55     if (currentMillis - previousMillis >= READ_INTERVAL)
56     {
57         // updates last exec time
58         previousMillis = currentMillis;
59
60         readBarometer();
61         printSerial();
62         updateLCD();
63     }
64 }
65
66 void readBarometer()
67 {
68     // read temperature first
69     // BMP085 library requires temperature reading before calculating pressure
70     temperature = barometer.bmp085GetTemperature(barometer.bmp085ReadUT());
71
72     // read pressure in Pascals from the BMP085 sensor
73     pressurePa = barometer.bmp085GetPressure(barometer.bmp085ReadUP());
74
75     // convert (Pa to atm)
76     pressureAtm = pressurePa / PA_PER_ATM;
77
78     // estimate altitude based on pressure
79     altitude = barometer.calcAltitude(pressurePa);
80 }
81
82 void printSerial()
83 {
84     // print values to the SM
85     Serial.print("Temperature: ");
86     Serial.print(temperature);
87     Serial.print(" C | Pressure: ");
88     Serial.print(pressurePa);
89     Serial.print(" Pa | Pressure: ");
90     Serial.print(pressureAtm, 4);
91     Serial.print(" atm | Altitude: ");
```

```
91     Serial.print(" atm | Altitude: ");
92     Serial.print(altitude);
93     Serial.println(" m");
94 }
95
96 void updateLCD()
97 {
98     // LCD has only not enough columns/rows for everything
99     // only main exercise results are displayed (pressure in atm + altitude)
100    lcd.clear();
101
102    lcd.setCursor(0, 0);
103    lcd.print("P:");
104    lcd.print(pressureAtm, 4);
105    lcd.print(" atm");
106
107    lcd.setCursor(0, 1);
108    lcd.print("Alt:");
109    lcd.print(altitude);
110    lcd.print("m");
111 }
```

4. Termistor TTC03: Crear un sketch que imprima por pantalla las temperaturas obtenidas del termistor y del sensor DHT22 para realizar una comparación de ambas

Explicación

El objetivo de este ejercicio es comparar la temperatura obtenida mediante el sensor DHT22 con la lectura obtenida mediante el termistor TTC03.

El **DHT22** es un **sensor digital** de temperatura y humedad, por lo que proporciona directamente la temperatura en grados Celsius. En cambio, el termistor **TTC03** es un **sensor analógico** cuya lectura se obtiene mediante una entrada analógica de Arduino, utilizamos `'analogRead()'` y posteriormente convertimos a grados Celsius utilizando la ecuación de **Steinhart-Hart**.

El programa inicializa el sensor DHT22, la pantalla LCD I2C y la comunicación serie (SM). Cada 2 segundos se leen las temperaturas proporcionadas por ambos sensores. Los resultados se muestran tanto en la pantalla LCD como en el Serial Monitor (SM) para poder compararlos. Como en los ejercicios anteriores, se utiliza *millis* en lugar de *delay* para controlar el tiempo entre lecturas sin bloquear la ejecución del programa. Además, la pantalla LCD solo se actualiza cuando los valores cambian.

Conversión usada

1. Cálculo de la resistencia

La resistencia del termistor se obtiene a partir de la lectura analógica utilizando la ecuación del divisor de tensión:

$$R = ((1023 - SEÑAL) \cdot \frac{R_SERIE}{SEÑAL})$$

- SEÑAL → valor leído del ADC (0–1023)
- R_SERIE → resistencia conocida en serie
- R → resistencia del sensor (termistor)

2. Cálculo de la temperatura (ecuación de Steinhart-Hart)

La resistencia se convierte a temperatura mediante la ecuación de Steinhart-Hart:

$$T = \frac{1}{\frac{\ln(\frac{R}{R_{NOM}})}{B} + \frac{1}{(T_{NOM} + 273.15)}} - 273.15$$

- $R \rightarrow$ resistencia calculada
- $R_{NOM} \rightarrow$ resistencia nominal del termistor
- $B \rightarrow$ coeficiente Beta del termistor
- $T_{NOM} \rightarrow$ temperatura nominal en °C
- $T \rightarrow$ temperatura en °C

Steinhart Hart Equation

$$\frac{1}{T} = A + B \ln(R) + C [\ln(R)]^3$$

Resultado

La pantalla LCD muestra la temperatura medida por el termistor TTC03 y la temperatura medida por el DHT22. Además, ambos valores se imprimen en el SM para facilitar la comparación entre sensores. En este caso, las temperaturas obtenidas por ambos sensores son muy similares, aunque pueden existir pequeñas variaciones debido a las diferencias de precisión, calibración y funcionamiento entre un sensor analógico y uno digital.



```
Thermistor: 16.71 C | DHT22: 17.90 C
Thermistor: 16.71 C | DHT22: 17.90 C
Thermistor: 16.71 C | DHT22: 17.80 C
Thermistor: 16.71 C | DHT22: 17.90 C
```

Código

```

1  *
2  Exercise 4 - Thermistor TTC03 and DHT22 comparison
3
4  This program reads temperature from TTC03 (thermistor) and DHT22 sensor
5  Both values are displayed on both the LCD and SM
6
7  -> The thermistor is an analog sensor, so its reading is converted
8  to temperature using the Steinhart-Hart equation
9
10 -> The DHT22 is a digital sensor and directly provides temperature
11 in Celsius
12 /
13
14 include <Wire.h>           // I2C communication library
15 include <LiquidCrystal_I2C.h> // LCD display library
16 include "DHT.h"           // DHT sensor library
17 include <math.h>           // math library used for logarithm
18
19 / thermistor config
20 define THERMISTOR_PIN A2
21
22 define R_NOMINAL 10000
23 define TEMP_NOMINAL 25
24 define B_COEFF 3435
25 define R_SERIE 10000
26
27 / DHT22 config
28 define DHT22_PIN 12
29 define DHTTYPE DHT22
30
31 #HT dht(DHT22_PIN, DHTTYPE);
32
33 / LCD object: address 0x3F, 16 columns, 2 rows
34 LiquidCrystal_I2C lcd(0x3F, 16, 2);
35
36 / timing config
37 const unsigned long READ_INTERVAL = 2000;
38 unsigned long previousMillis = 0;
39
40 / sensor values
41 float thermistorTemperature = 0;
42 float dhtTemperature = 0;
43
44 / last displayed values, used to avoid unnecessary LCD refreshes
45 float lastThermistorTemp = -1000;
46 float lastDhtTemp = -1000;

```

```
46 float lastDhtTemp = -1000;
47
48 void setup()
49 {
50     // init SM
51     Serial.begin(9600);
52
53     // init LCD
54     lcd.init();
55     lcd.backlight();
56     lcd.clear();
57
58     // Init DHT22
59     dht.begin();
60
61     // startup message
62     lcd.setCursor(0, 0);
63     lcd.print("Temp compare");
64 }
65
66 void loop()
67 {
68     // get current time since program started
69     unsigned long currentMillis = millis();
70
71     // check if defined time interval has passed
72     if (currentMillis - previousMillis >= READ_INTERVAL)
73     {
74         // updates last exec time
75         previousMillis = currentMillis;
76
77         // check if reading went OK
78         if (readSensors())
79         {
80             printSerial();
81             updateLCDIfNeeded();
82         }
83         else
84         {
85             handleSensorError();
86         }
87     }
88 }
89
```

```
90 bool readSensors()
91 {
92     // read thermistor temperature
93     thermistorTemperature = readThermistorTemperature();
94
95     // read DHT22 temperature in Celsius
96     dhtTemperature = dht.readTemperature();
97
98     // DHT22 returns NaN if reading fails
99     if (isnan(dhtTemperature))
100     {
101         return false;
102     }
103
104     return true;
105 }
106
107 float readThermistorTemperature()
108 {
109     // read raw analog value from thermistor
110     float signal = analogRead(THERMISTOR_PIN);
111
112     // avoid division by zero (if analog reading = invalid)
113     if (signal <= 0)
114     {
115         return -1000;
116     }
117
118     // convert analog signal into resistance using voltage divider formula
119     float resistance = (float)(1023 - signal) * R_SERIE / signal;
120
121     // convert resistance into temperature using Steinhart-Hart equation
122     float temperature = 1.0 / (log(resistance / R_NOMINAL) / B_COEFF + 1.0 / (TEMP_NOMINAL + 273.15)) - 273.15;
123
124     return temperature;
125 }
126
127 void printSerial()
128 {
129     // print both temperatures to compare sensors
130     Serial.print("Thermistor: ");
131     Serial.print(thermistorTemperature);
132     Serial.print(" C | DHT22: ");
133     Serial.print(dhtTemperature);
134     Serial.println(" C");
135 }
136
```

```
137 void updateLCDIfNeeded()
138 {
139     // update LCD only if values changed
140     // -> This reduces flickering and unnecessary refreshes
141     if (abs(thermistorTemperature - lastThermistorTemp) >= 0.1 ||
142         abs(dhtTemperature - lastDhtTemp) >= 0.1)
143     {
144         lcd.setCursor(0, 0);
145         lcd.print("TTC:");
146         lcd.print(thermistorTemperature, 1);
147         lcd.print("C  ");
148
149         lcd.setCursor(0, 1);
150         lcd.print("DHT:");
151         lcd.print(dhtTemperature, 1);
152         lcd.print("C  ");
153
154         lastThermistorTemp = thermistorTemperature;
155         lastDhtTemp = dhtTemperature;
156     }
157 }
158
159 void handleSensorError()
160 {
161     // error message if DHT22 reading fails
162     Serial.println("Error: DHT22 reading failed");
163
164     lcd.setCursor(0, 0);
165     lcd.print("Sensor error  ");
166     lcd.setCursor(0, 1);
167     lcd.print("Check DHT22  ");
168 }
```

5. Sensor de luz GL5528: Hacer girar el servo hasta posicionar la pestaña encima del sensor, en esta posición apuntar el valor de la intensidad de luz obtenido. Crear un sketch que haga girar el servo desde su posición inicial y que además lea el valor dado por el sensor, de forma que haga parar el servo en el momento en el que la lectura corresponde con el valor previamente obtenido

Explicación

El objetivo de este ejercicio es controlar un **servomotor** en función de la lectura de un **sensor de luz**, de manera que el servo **se detenga cuando se alcanza un valor de iluminación** previamente definido.

El **sensor** de luz proporciona un **valor analógico** que depende de la intensidad luminosa. El programa lee continuamente este valor y lo compara con un valor objetivo previamente obtenido. Para evitar problemas derivados de pequeñas variaciones en la lectura, se utiliza una **tolerancia**, de modo que el servo se detiene cuando el valor de luz se encuentra dentro de un **rango cercano al valor objetivo**.

El movimiento del servo se realiza de forma progresiva, aumentando su posición paso a paso hasta encontrar el valor adecuado. Mientras tanto, los valores del sensor y la posición del servo se muestran en el Serial Monitor (SM) y en la pantalla LCD.

Para mejorar la estructura del código, se han utilizado funciones independientes para la lectura del sensor, el control del servo, la impresión de los datos y la actualización del LCD. Además, se utiliza *millis* en lugar de *delay* para evitar bloquear la ejecución del programa, permitiendo un control más eficiente del sistema.

Resultado

Al ejecutar el programa, el servomotor comienza a moverse progresivamente mientras se leen los valores del sensor de luz. Estos valores se muestran en el Serial Monitor y en la pantalla LCD.

Cuando la lectura del sensor se aproxima al valor objetivo dentro de la tolerancia establecida, el servo se detiene automáticamente. En este caso, el valor objetivo utilizado es 435, con una tolerancia de ± 10 , por lo que el servo se detiene cuando la lectura se encuentra entre 425 y 445.

En la ejecución realizada, el servo se detuvo en la posición 0° al obtener una lectura de luz de 430. Como el valor objetivo era 435 y la tolerancia establecida era ± 10 , la lectura se encuentra dentro del rango válido.



```
light: 453 | servo: 134  
light: 452 | servo: 135  
light: 452 | servo: 136  
light: 452 | servo: 137  
light: 450 | servo: 138  
light: 452 | servo: 1
```



```
target reached | light: 430 | servo stopped at: 0
```

Código

```
1  /*
2   Exercise 5 - GL5228 Light sensor and Servo
3
4   This program rotates a servo while reading the light sensor
5   If sensor value matches target value-> servo finds light -> servo stops moving
6
7   Current light value is displayed on both the LCD and Serial Monitor
8  */
9
10 #include <Wire.h>           // I2C communication library
11 #include <LiquidCrystal_I2C.h> // LCD display library
12 #include <Servo.h>         // Servo library
13
14 // light sensor config
15 #define LIGHT_SENSOR_PIN A15
16
17 // servo config
18 #define SERVO_PIN 41
19
20 // target value (this must be measured manually before)
21 #define TARGET_LIGHT_VALUE 435
22 // error range (allowed)
23 #define TOLERANCE 10
24
25 // LCD object: address 0x3F, 16 columns, 2 rows
26 LiquidCrystal_I2C lcd(0x3F, 16, 2);
27
28 // servo obj
29 Servo myservo;
30
31 // timing config
32 const unsigned long READ_INTERVAL = 500;
33 unsigned long previousMillis = 0;
34
35 // light variables
36 int lightValue = 0;
37 int lastLightValue = -1;
38
39 // servo variables
40 int servoPos = 0;
41 bool targetReached = false;
42
43 void setup()
44 {
45     Serial.begin(9600);
46 }
```

```
47 // init LCD
48 lcd.init();
49 lcd.backlight();
50 lcd.clear();
51
52 // init light
53 pinMode(LIGHT_SENSOR_PIN, INPUT);
54
55 // init servo
56 myservo.attach(SERVO_PIN);
57
58 // startup message
59 lcd.setCursor(0, 0);
60 lcd.print("light search");
61 }
62
63 void loop()
64 {
65     unsigned long currentMillis = millis();
66
67     // control reading frequency
68     if (currentMillis - previousMillis >= READ_INTERVAL)
69     {
70         // update last exec time
71         previousMillis = currentMillis;
72
73         readLight();
74
75         if (!targetReached)
76         {
77             moveServo();
78         }
79
80         printSerial();
81         updateLCDIfNeeded();
82     }
83 }
84
85 // read current light value
86 void readLight()
87 {
88     lightValue = analogRead(LIGHT_SENSOR_PIN);
89 }
90
91 void moveServo()
92 {
```

```
91 void moveServo()  
92 {  
93     // check if current value is close to target  
94     if (abs(lightValue - TARGET_LIGHT_VALUE) <= TOLERANCE)  
95     {  
96         targetReached = true;  
97         Serial.println("target reached");  
98         return;  
99     }  
100  
101     // move servo step by step  
102     servoPos++;  
103  
104     if (servoPos > 180)  
105     {  
106         servoPos = 0;  
107     }  
108  
109     myservo.write(servoPos);  
110 }  
111  
112 // print on Serial Monitor  
113 void printSerial()  
114 {  
115     if (targetReached)  
116     {  
117         Serial.print("target reached | light: ");  
118         Serial.print(lightValue);  
119         Serial.print(" | servo stopped at: ");  
120         Serial.println(servoPos);  
121     }  
122     else  
123     {  
124         Serial.print("light value: ");  
125         Serial.print(lightValue);  
126         Serial.print(" | servo position: ");  
127         Serial.println(servoPos);  
128     }  
129 }  
130
```

```
131 void updateLCDIfNeeded()
132 {
133     // print only if target reached
134     if (targetReached)
135     {
136         lcd.setCursor(0, 0);
137         lcd.print("target found   ");
138         lcd.setCursor(0, 1);
139         lcd.print("servo stopped   ");
140         return;
141     }
142
143     // update lcd only if value changes
144     if (lightValue != lastLightValue)
145     {
146         lcd.setCursor(0, 0);
147         lcd.print("light:");
148         lcd.print(lightValue);
149         lcd.print("   ");
150
151         lcd.setCursor(0, 1);
152         lcd.print("searching...   ");
153
154         lastLightValue = lightValue;
155     }
156 }
```

6. Sensor RFID RC522: Modificar el sketch de ejemplo para que el servo se posicione sobre el lector de RFID y anotar el valor del número de serie de la tarjeta TAG

Explicación

El objetivo de este ejercicio es posicionar el servomotor sobre el lector RFID RC522 y leer el número de serie de la tarjeta TAG.

Para ello, utilizamos **el lector RFID RC522** junto con un **servomotor**. El servo se mueve hasta una posición fija, definida en el programa, para **situar la tarjeta TAG cerca del lector RFID**. Una vez posicionada, el programa comprueba si hay una tarjeta disponible y, en caso afirmativo, **lee su número de serie o UID**.

El lector RFID RC522 se comunica con Arduino mediante el protocolo **SPI** (*Serial Peripheral Interface*).¹

El programa inicializa el servo, el lector RFID, la pantalla LCD I2C y el Serial Monitor (SM). Cuando se detecta una tarjeta, se obtiene su UID byte a byte, se convierte a formato hexadecimal y se muestra en el SM y en la pantalla LCD

¹ Protocolo de comunicación serie síncrona utilizado para intercambiar datos de alta velocidad entre Arduino y dispositivos periféricos como sensores, lectores RFID o memorias.

Resultado

Al ejecutar el programa, el servo se posiciona sobre el lector RFID. Cuando el lector detecta la tarjeta TAG, se obtiene su UID y se muestra en el SM. En la pantalla LCD se indica que se ha detectado una tarjeta y se muestra el UID leído. Debido al tamaño limitado de la pantalla LCD 16x2, si el UID es demasiado largo puede mostrarse parcialmente.



RFIDRFID tag UID: 8A:54:D6:80

Código

```
1  /*
2   Exercise 6 - RFID RC522 and servo
3
4   This program moves the servo to place the RFID tag near the reader.
5   Then, reads the RFID tag UID and displays it both on the LCD and Serial Monitor
6  */
7
8  #include <Wire.h>           // I2C communication library
9  #include <LiquidCrystal_I2C.h> // LCD display library
10 #include <Servo.h>          // Servo motor library
11 #include <SPI.h>            // SPI communication library
12 #include <MFRC522.h>        // RFID RC522 library
13
14 // servo config
15 #define SERVO_PIN 41
16 // manual RFID position measured in the lab
17 #define RFID_POSITION 180
18
19 // RFID config
20 // SS_PIN 53 is used because Arduino Mega uses pin 53 as SPI SS
21 #define SS_PIN 53
22 #define RST_PIN 5
23
24 // hardware objects
25 Servo myservo;
26 MFRC522 rfid(SS_PIN, RST_PIN);
27 LiquidCrystal_I2C lcd(0x3F, 16, 2);
28
29 // tag variables
30 String tagUID = "";
31 bool tagRead = false;
32
33 void setup()
34 {
35     // init Serial communication
36     Serial.begin(9600);
37
38     // init LCD
39     lcd.init();
40     lcd.backlight();
41     lcd.clear();
42
43     // init servo (attached)
44     myservo.attach(SERVO_PIN);
45
46     // init RFID communication
```

```
46 // init RFID communication
47 SPI.begin();
48 rfid.PCD_Init();
49
50 // move servo to RFID reader pos
51 myservo.write(RFID_POSITION);
52
53 // startup message
54 lcd.setCursor(0, 0);
55 lcd.print("rfid ready");
56 lcd.setCursor(0, 1);
57 lcd.print("scan tag...");
58 }
59
60 void loop()
61 {
62     // if the tag has already been read, keep the result on screen
63     if (tagRead)
64     {
65         return;
66     }
67
68     readRFID();
69 }
70
71 void readRFID()
72 {
73     // check if a new RFID card is present
74     if (!rfid.PICC_IsNewCardPresent())
75     {
76         return;
77     }
78
79     // try to read card serial number
80     if (!rfid.PICC_ReadCardSerial())
81     {
82         return;
83     }
84
85     tagUID = "";
86
87     // build UID string from card bytes
88     for (byte i = 0; i < rfid.uid.size; i++)
89     {
90         if (rfid.uid.uidByte[i] < 0x10)
91         {
```

```
90     if (rfid.uid.uidByte[i] < 0x10)
91     {
92         tagUID += "0";
93     }
94
95     tagUID += String(rfid.uid.uidByte[i], HEX);
96
97     if (i < rfid.uid.size - 1)
98     {
99         tagUID += ":";
100     }
101 }
102
103 // convert UID to uppercase
104 tagUID.toUpperCase();
105
106 // mark tag as read
107 tagRead = true;
108
109 printSerial();
110 updateLCD();
111
112 // stop communication with current card
113 rfid.PICC_HaltA();
114 }
115
116 void printSerial()
117 {
118     // print RFID tag uid to Serial Monitor
119     Serial.print("rfid tag uid: ");
120     Serial.println(tagUID);
121 }
122
123 void updateLCD()
124 {
125     // display UID on LCD
126     lcd.clear();
127
128     lcd.setCursor(0, 0);
129     lcd.print("tag detected");
130
131     lcd.setCursor(0, 1);
132     lcd.print(tagUID);
133 }
```

7. Sensor de movimiento PIR: Modificar el sketch de ejemplo para hacer girar el servo (en bucle) cerca del sensor y activar su salida

Explicación

El objetivo de este ejercicio es hacer girar el servomotor en bucle cerca del sensor PIR para activar su salida y comprobar la detección de movimiento. Para lograrlo, utilizamos un sensor PIR junto con un **servomotor**. El sensor **PIR** permite **detectar movimiento** mediante **radiación infrarroja**, generando una **salida digital**. Cuando detecta movimiento, su salida toma el valor **HIGH**; cuando no detecta movimiento, toma el valor **LOW**.

El programa mueve el servo de forma continua entre 0° y 180° para simular movimiento cerca del sensor. Al mismo tiempo, se lee el estado digital del PIR mediante la función '*digitalRead*'. El resultado se muestra en el Serial Monitor y en la pantalla LCD.

Como en ejercicios anteriores, se ha organizado el código en funciones independientes para separar la lectura del sensor, el movimiento del servo, la impresión de datos y la actualización de la pantalla. Además, se utiliza *millis* en lugar de *delay* para controlar el tiempo de ejecución sin bloquear el programa.

Resultado

Al ejecutar el programa, el servo comienza a moverse de forma repetitiva. Cuando el sensor PIR detecta dicho movimiento, el estado mostrado cambia a “**motion detected**”. En caso contrario, se muestra “**no motion**”. El resultado puede observarse tanto en el Serial Monitor (SM) como en la pantalla LCD.

```
pir: no motion | servo: 6
pir: motion detected | servo: 7
pir: motion detected | servo: 8
pir: no motion | servo: 9
pir: no motion | servo: 10
```



Código

```
1  ✓ /*
2      Exercise 7 - PIR motion sensor + servo
3
4      This program moves a servo continuously near the PIR sensor.
5      The PIR sensor detects motion and displays the result both on the LCD and SM
6  */
7
8  #include <Wire.h>           // I2C communication library
9  #include <LiquidCrystal_I2C.h> // LCD display library
10 #include <Servo.h>          // servo motor library
11
12 // pir sensor config
13 #define PIR_PIN 7
14
15 // servo config
16 #define SERVO_PIN 41
17
18 // lcd
19 LiquidCrystal_I2C lcd(0x3F, 16, 2);
20
21 // servo object
22 Servo myservo;
23
24 // timing config
25 const unsigned long READ_INTERVAL = 300;
26 unsigned long previousMillis = 0;
27
28 // PIR + Servo variables
29 int pirState = LOW;
30 int lastPirState = -1;
31 int servoPos = 0;
32 int servoDirection = 1;
33
34 void setup()
35 ✓ {
36     // init Serial communication
37     Serial.begin(9600);
38     // startup message
39     Serial.println("PIR system ready");
40
41     // init LCD
42     lcd.init();
43     lcd.backlight();
44     lcd.clear();
45
46     // init PIR
```

```
46 // init PIR
47 pinMode(PIR_PIN, INPUT);
48
49 // init servo
50 myservo.attach(SERVO_PIN);
51 myservo.write(servoPos);
52
53 // startup message
54 lcd.setCursor(0, 0);
55 lcd.print("pir ready");
56 }
57
58 void loop()
59 {
60 // get current time since program started
61 unsigned long currentMillis = millis();
62
63 // check if defined time interval has passed
64 if (currentMillis - previousMillis >= READ_INTERVAL)
65 {
66 //update previous time variable
67 previousMillis = currentMillis;
68
69 // read PIR sensor state
70 readPIR();
71 //move servo continuously
72 moveServo();
73 printSerial();
74 updateLCDIfNeeded();
75 }
76 }
77
78 void readPIR()
79 {
80 // read PIR digital output
81 // -> high means motion detected, low means no motion
82 pirState = digitalRead(PIR_PIN);
83 }
84
85 void moveServo()
86 {
87 // move servo back and forth between 0 and 180 degrees
88 servoPos += servoDirection;
89
90 // change direction when reaching maximum angle
91 if (servoPos >= 180)
```

```
90 // change direction when reaching maximum angle
91 if (servoPos >= 180)
92 {
93     servoPos = 180;
94     servoDirection = -1;
95 }
96 // change direction when reaching min angle
97 else if (servoPos <= 0)
98 {
99     servoPos = 0;
100     servoDirection = 1;
101 }
102
103 // send new position to servo
104 myservo.write(servoPos);
105 }
106
107 void printSerial()
108 {
109     // print motion status and servo position
110     Serial.print("pir: ");
111
112     if (pirState == HIGH)
113     {
114         Serial.print("motion detected");
115     }
116     else
117     {
118         Serial.print("no motion");
119     }
120
121     Serial.print(" | servo: ");
122     Serial.println(servoPos);
123 }
124
125 void updateLCDIfNeeded()
126 {
127     // display PIR status on first LCD row
128     lcd.setCursor(0, 0);
129
130     if (pirState == HIGH)
131     {
132         lcd.print("motion detected ");
133     }
134     else
135     {
136         lcd.print("no motion      ");
137     }
138 }
```

```
137     }  
138  
139     // display current servo position on second row  
140     lcd.setCursor(0, 1);  
141     lcd.print("servo:");  
142     lcd.print(servoPos);  
143     lcd.print("  ");  
144  
145     lastPirState = pirState;  
146 }
```

8. Sensor de sonido: Modificar el código de ejemplo para muestrear el sensor de sonido 'N_SAMPLES' veces (poner 10 por defecto) y calcular el valor medio, a continuación, comparar el valor medio con el umbral

Explicación

El objetivo de este ejercicio es leer el sensor de sonido varias veces, calcular un valor medio y compararlo con un umbral definido. Para ello, el programa toma **N_SAMPLES** muestras del sensor, configurado con valor **10 por defecto**, suma todas las lecturas y calcula la media.

El **sensor de sonido** proporciona una **lectura analógica** mediante '*analogRead*'. En lugar de usar una única lectura, se calcula el **promedio de varias muestras** para obtener un valor más **estable** y reducir el efecto de posibles variaciones puntuales.

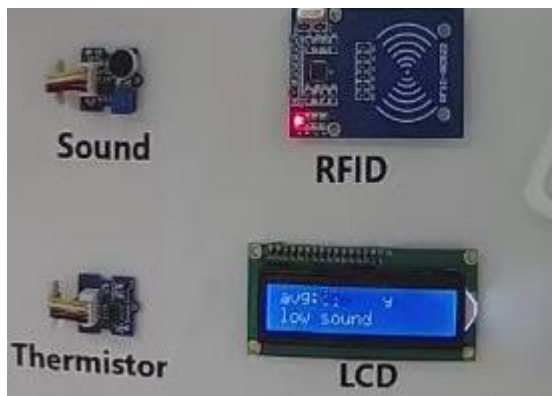
Después, el valor medio **se compara con un umbral** (**SOUND_THRESHOLD**). Si la media supera dicho umbral, el sistema considera que hay un nivel de sonido **alto**. Si no lo supera, se considera que el sonido es **bajo**.

Como en los ejercicios anteriores, el código se organiza en funciones independientes y se utiliza *millis* en lugar de *delay* para controlar el tiempo de lectura sin bloquear la ejecución del programa.

Resultado

Al ejecutar el programa, se muestra en el Serial Monitor (SM) y en la pantalla LCD el valor medio del sonido calculado a partir de $N_SAMPLES = 10$ muestras. Además, el sistema indica si el nivel de sonido detectado supera o no el umbral establecido.

En las pruebas realizadas, los valores medios obtenidos estuvieron aproximadamente entre 90 y 201. Inicialmente se utilizó un umbral de 500, pero se observó que era demasiado alto para las condiciones del entorno. Por ello, en la versión final del código se ajustó el umbral a 200, de forma que valores cercanos o superiores a 200 se clasifican como sonido alto.



```
sound average: 90 | status: low sound  
sound average: 94 | status: low sound  
sound average: 90 | status: low sound  
sound average: 201 | status: low sound
```

Código

```

1  ✓ /*
2      Exercise 8 - Sound sensor
3
4      This program reads the sound sensor several times, calculates
5      the average value, and compares it with a defined threshold.
6
7      -> The average sound value and sound status are displayed on
8      the LCD and Serial Monitor.
9  */
10
11  #include <Wire.h>           // I2C communication library
12  #include <LiquidCrystal_I2C.h> // LCD display library
13
14  // analog input pin used to read the sound sensor signal
15  #define SOUND_SENSOR_PIN A0
16  // sound threshold used to classify sound level
17  #define SOUND_THRESHOLD 500
18  // number of samples used to calculate the average value
19  #define N_SAMPLES 10
20
21  // LCD object: address 0x3F, 16 columns, 2 rows
22  LiquidCrystal_I2C lcd(0x3F, 16, 2);
23
24  // timing config
25  const unsigned long READ_INTERVAL = 1000;
26  unsigned long previousMillis = 0;
27
28  // sound variables
29  int soundAverage = 0;
30  int lastSoundAverage = -1;
31  bool soundDetected = false;
32
33  void setup()
34  ✓ {
35      // init Serial communication
36      Serial.begin(9600);
37
38      // init LCD
39      lcd.init();
40      lcd.backlight();
41      lcd.clear();
42
43      // init sound sensor
44      pinMode(SOUND_SENSOR_PIN, INPUT);
45
46      // startup message

```

```
46     // startup message
47     lcd.setCursor(0, 0);
48     lcd.print("sound ready");
49 }
50
51 void loop()
52 {
53     // get current time since program started
54     unsigned long currentMillis = millis();
55
56     // check if defined time interval has passed
57     if (currentMillis - previousMillis >= READ_INTERVAL)
58     {
59         //update previous time variable
60         previousMillis = currentMillis;
61
62         // read average sound value
63         readSoundAverage();
64         // compare average value with threshold
65         compareWithThreshold();
66         printSerial();
67         updateLCDIfNeeded();
68     }
69 }
70
71 // read the sound sensor N_SAMPLES times and calculate the average value
72 void readSoundAverage()
73 {
74
75     long sum = 0;
76
77     for (int i = 0; i < N_SAMPLES; i++)
78     {
79         sum += analogRead(SOUND_SENSOR_PIN);
80     }
81
82     // calculate average value
83     soundAverage = sum / N_SAMPLES;
84 }
85
86 void compareWithThreshold()
87 {
88     // compare average value with threshold
89     // >= threshold -> high sound detected
90     if (soundAverage >= SOUND_THRESHOLD)
91     {
```

```
92     soundDetected = true;
93 }
94 else
95 {
96     soundDetected = false;
97 }
98 }
99
100 void printSerial()
101 {
102     // print average value and sound status to SM
103     Serial.print("sound average: ");
104     Serial.print(soundAverage);
105     Serial.print(" | status: ");
106
107     if (soundDetected)
108     {
109         Serial.println("high sound");
110     }
111     else
112     {
113         Serial.println("low sound");
114     }
115 }
116
117 void updateLCDIfNeeded()
118 {
119     // update LCD only if sound average changes
120     if (soundAverage != lastSoundAverage)
121     {
122         lcd.setCursor(0, 0);
123         lcd.print("avg:");
124         lcd.print(soundAverage);
125         lcd.print(" ");
126
127         // display sound status
128         lcd.setCursor(0, 1);
129
130         if (soundDetected)
131         {
132             lcd.print("high sound");
133         }
134         else
135         {
136             lcd.print("low sound");
137         }
138     }
139 }
```

```
138  
139     // store last displayed value  
140     lastSoundAverage = soundAverage;  
141 }  
142 }  
143
```

9. Sensor de gas MQ2: Simular e indicar los pasos necesarios en el proceso de calibración del sensor

Explicación

El objetivo de este ejercicio es simular e indicar los pasos necesarios para realizar el proceso de calibración del sensor de gas **MQ2**. Este sensor nos permite detectar distintos gases y humo mediante una **variación** en su **conductividad interna**. Cuando el sensor se encuentra en **aire limpio**, su **lectura analógica** tiene un valor de referencia. Si aparece una concentración de gas, la **conductividad** del sensor **cambia** y el **valor** leído por Arduino **aumenta**.

Para calibrar el sensor, primero se deben tomar varias lecturas en condiciones de aire limpio. A partir de esas muestras se calcula un valor medio, que se utiliza como línea base o referencia inicial. Después, se define un umbral superior añadiendo un margen a dicha línea base. Si posteriormente el valor leído supera ese umbral, se considera que existe presencia de gas.

Pasos de calibración

1. Encender el sensor MQ2 y dejar que se estabilice
2. Tomar varias lecturas en condiciones de aire limpio
3. Calcular la media de las lecturas obtenidas.
4. Utilizar esa media como valor base (*baseline*)
5. Definir un umbral añadiendo un margen de seguridad
(*THRESHOLD_OFFSET*)
6. Comparar las nuevas lecturas con el umbral definido
7. Si la lectura supera el umbral, considerar que existe presencia de gas

Resultado

Al ejecutar el programa, se toman varias muestras iniciales del sensor MQ2 para simular la calibración. Con estas muestras se calcula automáticamente un valor base de referencia y, posteriormente, un umbral de detección.

En las pruebas realizadas, las lecturas obtenidas fueron cercanas a 0, por lo que el valor base calculado también fue muy bajo y el umbral final quedó definido en 107. Las nuevas lecturas del sensor se comparan continuamente con dicho umbral para determinar si el aire se encuentra en condiciones normales o si existe una posible presencia de gas.

La información se muestra tanto en el SM como en la pantalla LCD, indicando el valor actual del sensor, el umbral calculado y el estado del sistema (“normal air” o “gas detected”)



```
7 | status: normal air
gas value: 0 | threshold: 107 | status: normal air
gas value: 0 | threshold: 107 | status: normal air
7 | status: normal air
gas value: 0 | threshold: 107 | status: normal air
gas value: 0 | threshold: 107 | status: normal air
MQ2 calibration started
calibration sample 1: 0
```

Código

```
1  ✓ /*
2      Exercise 9 - MQ2 Gas sensor calibration
3
4      This program simulates the calibration process of the MQ2 gas sensor
5
6      It reads several analog values in clean air, calculates the average,
7      and uses that value as a reference baseline.
8
9      This baseline can later be used to define a detection threshold.
10 */
11
12 #include <Wire.h>           // I2C communication library
13 #include <LiquidCrystal_I2C.h> // LCD display library
14
15 // MQ2 sensor config
16 #define MQ2_PIN A1
17 #define N_SAMPLES 20
18 #define THRESHOLD_OFFSET 100
19
20 // LCD object: address 0x3F, 16 columns, 2 rows
21 LiquidCrystal_I2C lcd(0x3F, 16, 2);
22
23 // timing configuration
24 const unsigned long READ_INTERVAL = 1000;
25 unsigned long previousMillis = 0;
26
27 // gas variables
28 int baseline = 0;
29 int threshold = 0;
30 int gasValue = 0;
31 int lastGasValue = -1;
32
33 void setup()
34 {
35     // init Serial communication
36     Serial.begin(9600);
37     Serial.println("MQ2 calibration started");
38
39     // init LCD
40     lcd.init();
41     lcd.backlight();
42     lcd.clear();
43
44     // init MQ2
45     pinMode(MQ2_PIN, INPUT);
46     lcd.setCursor(0, 0);
```

```
47 // debug
48 lcd.print("mq2 calibrating");
49
50 // calibrate MQ2 sensor in clean air
51 calibrateMQ2();
52
53 // show calibration result on LCD
54 lcd.clear();
55 lcd.setCursor(0, 0);
56 lcd.print("baseline:");
57 lcd.print(baseline);
58
59 lcd.setCursor(0, 1);
60 lcd.print("th:");
61 lcd.print(threshold);
62 }
63
64 void loop()
65 {
66 // get current time since program started
67 unsigned long currentMillis = millis();
68 // check if defined time interval has passed
69 if (currentMillis - previousMillis >= READ_INTERVAL)
70 {
71 // Update previous execution time
72 previousMillis = currentMillis;
73
74 // Read current gas sensor value
75 readGas();
76
77 // Print values to Serial Monitor
78 printSerial();
79
80 // Update LCD display
81 updateLCDIfNeeded();
82 }
83 }
84
85 void calibrateMQ2()
86 {
87 long sum = 0;
88
89 // take several readings in clean air
90 for (int i = 0; i < N_SAMPLES; i++)
91 {
```

```
92     int value = analogRead(MQ2_PIN);
93     sum += value;
94
95     Serial.print("calibration sample ");
96     Serial.print(i + 1);
97     Serial.print(": ");
98     Serial.println(value);
99
100    delay(200);
101  }
102
103  // calculate clean air baseline
104  baseline = sum / N_SAMPLES;
105
106  // define a threshold above the baseline
107  threshold = baseline + THRESHOLD_OFFSET;
108
109  Serial.print("baseline: ");
110  Serial.println(baseline);
111
112  Serial.print("threshold: ");
113  Serial.println(threshold);
114  }
115  void readGas()
116  {
117    // read current MQ2 analog value
118    gasValue = analogRead(MQ2_PIN);
119  }
120
121  void printSerial()
122  {
123    // print gas value and threshold to Serial Monitor
124    Serial.print("gas value: ");
125    Serial.print(gasValue);
126    Serial.print(" | threshold: ");
127    Serial.print(threshold);
128    Serial.print(" | status: ");
129
130    if (gasValue > threshold)
131    {
132      Serial.println("gas detected");
133    }
134    else
135    {
136      Serial.println("normal air");
137    }
138  }
```

```
138     }
139
140     void updateLCDIfNeeded()
141     {
142         // update LCD only if gas value changes
143         if (gasValue != lastGasValue)
144         {
145             lcd.setCursor(0, 0);
146             lcd.print("gas:");
147             lcd.print(gasValue);
148             lcd.print("  ");
149
150             lcd.setCursor(0, 1);
151
152             if (gasValue > threshold)
153             {
154                 lcd.print("gas detected  ");
155             }
156             else
157             {
158                 lcd.print("normal air  ");
159             }
160
161             lastGasValue = gasValue;
162         }
163     }
164
```

10. Sensor de ultrasonidos HC-SR04: Indicar las diferentes distancias que indica el sensor cuando el servo está a 0°, 90° y 180°, teniendo en cuenta la precisión de este

Explicación

El objetivo de este ejercicio es medir la distancia detectada por el sensor de ultrasonidos **HC-SR04** cuando el **servomotor** se sitúa en tres posiciones diferentes: 0°, 90° y 180°. El sensor HC-SR04 calcula la distancia emitiendo una **señal ultrasónica** y **midiendo el tiempo que tarda el eco en regresar**. A partir de ese tiempo, se calcula la distancia utilizando la velocidad del sonido. Como la señal viaja hasta el objeto y vuelve al sensor, el resultado se divide entre dos.

La fórmula utilizada es:

$$D = \frac{T_{ECO} \cdot V_{SOUND}}{2}$$

- D es la distancia
- V_{SOUND} es la velocidad del sonido, aproximadamente 0.0343 cm/us
- T_{ECO} es el tiempo que tarda el eco en regresar

El programa mueve el servo secuencialmente a 0°, 90° y 180°. En cada posición se realiza una medición de distancia, que se muestra en el Serial Monitor y en la pantalla LCD.

Resultado

Al ejecutar el programa, el servomotor se mueve secuencialmente a las posiciones de 0°, 90° y 180°. En cada posición, el sensor ultrasónico HC-SR04 realiza una medición de distancia y muestra el resultado tanto en el SM como en la pantalla LCD.

En este laboratorio, el sensor ultrasónico HC-SR04 permanece fijo y orientado hacia el exterior del panel, mientras que el servomotor únicamente mueve el brazo mecánico blanco y no el propio sensor. Debido a ello, durante las pruebas el sensor no tenía un objeto cercano situado directamente enfrente desde el cual recibir el eco ultrasónico. Como consecuencia, ‘*pulseIn()*’ alcanzó el tiempo límite de espera (*timeout*) y el programa devolvió una distancia de 0.00 cm en todas las posiciones del servo.

Los valores obtenidos son anotados a continuación:

Posición del servo	Distancia medida
0°	0 cm
90°	0 cm
180°	0 cm

El sensor HC-SR04 tiene una precisión aproximada de ± 3 mm, por lo que las medidas obtenidas pueden presentar pequeñas variaciones.



```
servngle: 0 degrees | distance: 0.00 cm
servo angle: 90 degrees | distance: 0.00 cm
servo angle: 180 degrees | distance: 0.00 cm
angle: 0 degrees | distance: 0.00 cm
servo angle: 90 degrees | distance: 0.00 cm
servo angle: 180 degrees | distance: 0.00 cm
```

Código

```

1  ✓ /*
2      Exercise 10 - Ultrasonic Sensor HC-SR04 + servo
3
4      This program moves the servo to 0, 90 and 180 degrees.
5      At each position, reads distance using the HC-SR04 sensor
6      and prints the result on both the LCD and Serial Monitor.
7  */
8
9  #include <Wire.h>           // I2C communication library
10 #include <LiquidCrystal_I2C.h> // LCD display library
11 #include <Servo.h>          // Servo motor library
12
13 // ultrasonic sensor configuration
14 #define TRIG_PIN 9
15 #define ECHO_PIN 10
16
17 // servo config
18 #define SERVO_PIN 41
19
20 // sound speed in cm/us
21 #define SOUND_SPEED_CM_US 0.0343
22
23 // max echo waiting time in microseconds
24 #define ECHO_TIMEOUT 30000
25
26 // LCD object: address 0x3F, 16 columns, 2 rows
27 LiquidCrystal_I2C lcd(0x3F, 16, 2);
28
29 // servo object
30 Servo myservo;
31
32 // servo positions to test
33 int positions[] = {0, 90, 180};
34 const int N_POSITIONS = 3;
35 int currentIndex = 0;
36
37 // timing config
38 const unsigned long READ_INTERVAL = 3000;
39 unsigned long previousMillis = 0;
40
41 // distance value in cm
42 float distanceCm = 0;
43
44 void setup()
45 ✓ {
46     // init Serial communication

```

```
46 // init Serial communication
47 Serial.begin(9600);
48
49 // init LCD
50 lcd.init();
51 lcd.backlight();
52 lcd.clear();
53
54 // configure ultrasonic sensor pins
55 pinMode(TRIG_PIN, OUTPUT);
56 pinMode(ECHO_PIN, INPUT);
57
58 // init servo
59 myservo.attach(SERVO_PIN);
60
61 // startup message
62 lcd.setCursor(0, 0);
63 lcd.print("ultrasonic");
64 lcd.setCursor(0, 1);
65 lcd.print("ready");
66 }
67
68 void loop()
69 {
70 // get current time since program started
71 unsigned long currentMillis = millis();
72
73 // check if defined time interval has passed
74 if (currentMillis - previousMillis >= READ_INTERVAL)
75 {
76 // update previous execution time
77 previousMillis = currentMillis;
78
79 // move servo to current test pos
80 moveServoToPosition();
81
82 // read distance at current servo pos
83 distanceCm = readDistance();
84
85 // print result to SM & update LCD display
86 printSerial();
87 updateLCD();
88
89 // move to next test pos
90 currentIndex++;
91
```

```
92     // restart sequence after last pos
93     if (currentIndex >= N_POSITIONS)
94     {
95         currentIndex = 0;
96     }
97 }
98 }
99
100 void moveServoToPosition()
101 {
102     // move servo to current test pos
103     myservo.write(positions[currentIndex]);
104
105     // wait until servo reaches selected pos
106     delay(500);
107 }
108
109 float readDistance()
110 {
111     // sends short trigger pulse to start ultrasonic measurement
112     digitalWrite(TRIG_PIN, LOW);
113     delayMicroseconds(2);
114
115     digitalWrite(TRIG_PIN, HIGH);
116     delayMicroseconds(10);
117     digitalWrite(TRIG_PIN, LOW);
118
119     // measure echo duration in microseconds
120     // -> timeout avoids blocking the program if no echo is received
121     long duration = pulseIn(ECHO_PIN, HIGH, ECHO_TIMEOUT);
122
123     // if no echo is received, return 0
124     if (duration == 0)
125     {
126         return 0;
127     }
128
129     // convert time into distance in cm
130     // distance = (echo time * sound speed) / 2
131     float distance = duration * SOUND_SPEED_CM_US / 2;
132
133     return distance;
134 }
135
136 void printSerial()
137 {
```

```
138     // print servo angle and measured distance
139     Serial.print("servo angle: ");
140     Serial.print(positions[currentIndex]);
141     Serial.print(" degrees | distance: ");
142     Serial.print(distanceCm);
143     Serial.println(" cm");
144 }
145
146 void updateLCD()
147 {
148     // display angle and distance on LCD
149     lcd.clear();
150
151     lcd.setCursor(0, 0);
152     lcd.print("angle:");
153     lcd.print(positions[currentIndex]);
154
155     lcd.setCursor(0, 1);
156     lcd.print("dist:");
157     lcd.print(distanceCm, 1);
158     lcd.print(" cm");
159 }
```

11. Reloj en tiempo real DS1307: Obtener el día, mes y año e imprimirlos junto a la hora por pantalla.

Explicación

El objetivo de este ejercicio es obtener la fecha y la hora mediante el **reloj en tiempo real DS1307** y mostrarlas por pantalla. El DS1307 es un **módulo RTC** que permite mantener la fecha y la hora incluso cuando el sistema se apaga, gracias a una batería externa. Este módulo se comunica con Arduino mediante el bus **I2C**.

El programa inicializa la pantalla LCD, el Serial Monitor y el módulo RTC. Cada segundo se leen los valores correspondientes al día, mes, año, hora, minuto y segundo.

Posteriormente, estos datos se muestran tanto en el SM como en la pantalla LCD.

Como en los ejercicios anteriores, se utiliza *millis* en lugar de *delay* para controlar el intervalo de lectura sin bloquear la ejecución del programa.

Resultado

Al ejecutar el programa, se muestra la fecha en formato '*día/mes/año*' y la hora en formato '*hora: minuto: segundo*'. Estos valores se actualizan cada segundo y pueden visualizarse tanto en el Serial Monitor (SM) como en la pantalla LCD. En la versión final, los valores de un solo dígito se muestran con un cero delante para mejorar la legibilidad, por ejemplo 06/05/2026 y 12:48:09.



```
date: 06/05/2026 | time: 12:53:59
date: 06/05/2026 | time: 12:54:00
date: 06/05/2026 | time: 12:54:01
date: 06/05/2026 | time: 12:54:03
date: 06/05/2026 | time: 12:54:04
```

Código

```
1  /*
2   Exercise 11 - RTC DS1307
3
4   This program reads date and time from DS1307 Real Time Clock (RTC)
5   Date and time are displayed on both the LCD and Serial Monitor
6  */
7
8  #include <Wire.h>           // I2C communication library
9  #include <LiquidCrystal_I2C.h> // LCD display library
10 #include <I2C_RTC.h>        // RTC library
11
12 // LCD object: address 0x3F, 16 columns, 2 rows
13 LiquidCrystal_I2C lcd(0x3F, 16, 2);
14
15 // RTC object
16 DS1307 rtc;
17
18 // timing config
19 const unsigned long READ_INTERVAL = 1000;
20 unsigned long previousMillis = 0;
21
22 // date and time variables
23 int dayValue = 0;
24 int monthValue = 0;
25 int yearValue = 0;
26 int hourValue = 0;
27 int minuteValue = 0;
28 int secondValue = 0;
29
30 void setup()
31 {
32   // init Serial communication
33   Serial.begin(9600);
34
35   // init LCD
36   lcd.init();
37   lcd.backlight();
38   lcd.clear();
39
40   // init RTC
41   rtc.begin();
42
43   // startup message
44   lcd.setCursor(0, 0);
45   lcd.print("RTC DS1307 ready");
46 }
```

```
47
48 void loop()
49 {
50     // get current time since program started
51     unsigned long currentMillis = millis();
52
53     // check if defined time interval has passed
54     if (currentMillis - previousMillis >= READ_INTERVAL)
55     {
56         //update previous time variable
57         previousMillis = currentMillis;
58
59         // read current date and time
60         readRTC();
61
62         // print values to Serial Monitor and LCD
63         printSerial();
64         updateLCD();
65     }
66 }
67
68 void readRTC()
69 {
70     // read current date and time from RTC
71     dayValue = rtc.getDay();
72     monthValue = rtc.getMonth();
73     yearValue = rtc.getYear();
74
75     hourValue = rtc.getHours();
76     minuteValue = rtc.getMinutes();
77     secondValue = rtc.getSeconds();
78 }
79
80 void print2Digits(Print &output, int value)
81 {
82     // print a leading zero if value has only one digit
83     if (value < 10)
84     {
85         output.print("0");
86     }
87
88     output.print(value);
89 }
90
91 void printSerial()
92 {
```

```
93     // print date to Serial Monitor
94     Serial.print("date: ");
95     print2Digits(Serial, dayValue);
96     Serial.print("/");
97     print2Digits(Serial, monthValue);
98     Serial.print("/");
99     Serial.print(yearValue);
100
101     // print time to Serial Monitor
102     Serial.print(" | time: ");
103     print2Digits(Serial, hourValue);
104     Serial.print(":");
105     print2Digits(Serial, minuteValue);
106     Serial.print(":");
107     print2Digits(Serial, secondValue);
108     Serial.println();
109 }
110
111 void updateLCD()
112 {
113     // display date on first row
114     lcd.setCursor(0, 0);
115     lcd.print("date:");
116     print2Digits(lcd, dayValue);
117     lcd.print("/");
118     print2Digits(lcd, monthValue);
119     lcd.print("/");
120     lcd.print(yearValue);
121     lcd.print(" ");
122
123     // display time on second row
124     lcd.setCursor(0, 1);
125     lcd.print("time:");
126     print2Digits(lcd, hourValue);
127     lcd.print(":");
128     print2Digits(lcd, minuteValue);
129     lcd.print(":");
130     print2Digits(lcd, secondValue);
131     lcd.print(" ");
132 }
```